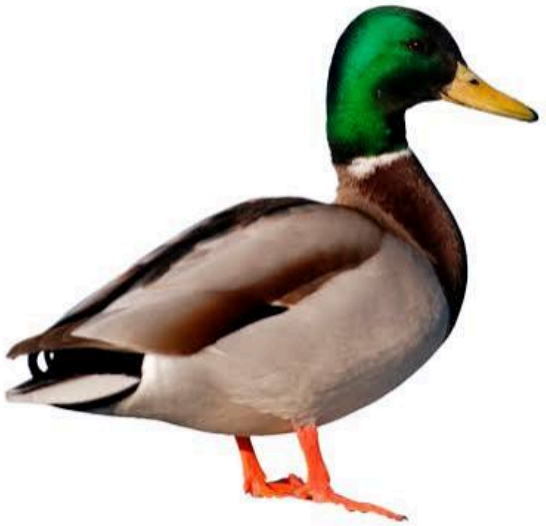
# Ciência de Dados Aplicada ao Direito I

## Aula 20

Modelos de linguagem



Prof. Bruno da Cunha de Oliveira - 2026/2

# Quem lembra vale ponto

Insper

# O que significam estas palavras?

**Deblaterar** **Estugar**

Escrevam o significado de cada uma. Cada palavra certa vale 0,1 ponto extra.

# Deblaterar

falar com veemência contra alguém ou algo; bradar

# Estugar

apressar o passo; estimular, incitar

# Os dois vocabulários do resto do curso

| Bibliotecas | Para quê |
|---|---|
| para usar IA: LangChain | pedir a um modelo de linguagem, de dentro do programa, que leia, classifique e extraia |
| para bases de dados: pandas e as de gráfico | ler, filtrar, juntar e resumir tabelas, e desenhar o resultado |

Em CDAD 2: conceitos de estatística e as bibliotecas de estatística.

# As licenças que vimos

# Licença: o que se pode fazer com o código

| Licença | O que permite, e o que pede em troca |
|---|---|
| MIT | usar, mudar, vender; manter o aviso e a licença |
| Apache 2.0 | o mesmo, e dizer o que foi mudado |
| BSD 3-Clause | o mesmo, sem usar o nome dos autores para promover |
| GPL e AGPL | quem distribui versão mudada abre o código; na AGPL, também quem a oferece pela internet |
| proprietária | só quem o dono autorizar usa, mesmo com o código à vista |
| nenhuma | o mesmo que a proprietária |

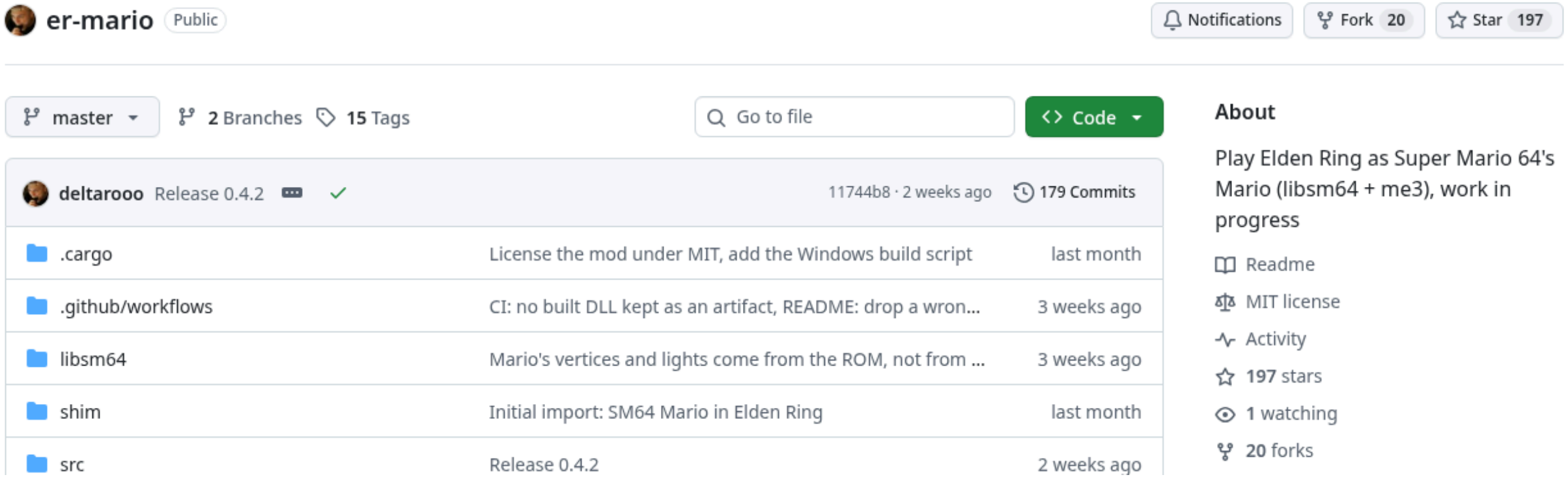
# A licença do grupo: três caminhos

| Caminho | Como fica |
|---|---|
| aberto | seguir com a licença da base, ou escolher uma no GitHub: **Add file**, **Create new file**, o nome `LICENSE` e o botão **Choose a license template** |
| fechado | licença proprietária: o repositório continua público para a banca, e só quem o grupo autorizar usa; o `LICENSE` diz "Copyright 2026 Grupo X. Todos os direitos reservados." |
| misto | uma parte aberta e outra fechada |

# O Mario em outros jogos

Animação no navegador: `animacao-video-mario.html`.

# Mario no Elden Ring, setembro de 2026




Fonte: github.com/deltarooo/er-mario, 26/10/2026

# Posso vender?

O repositório do mod do Elden Ring tem licença MIT, que deixa copiar, mudar e vender.

Posso pegar esse Mario, pôr num jogo meu e vender o jogo? Cada um escreve sim ou não, e por quê.

# O que a MIT do mod cobre

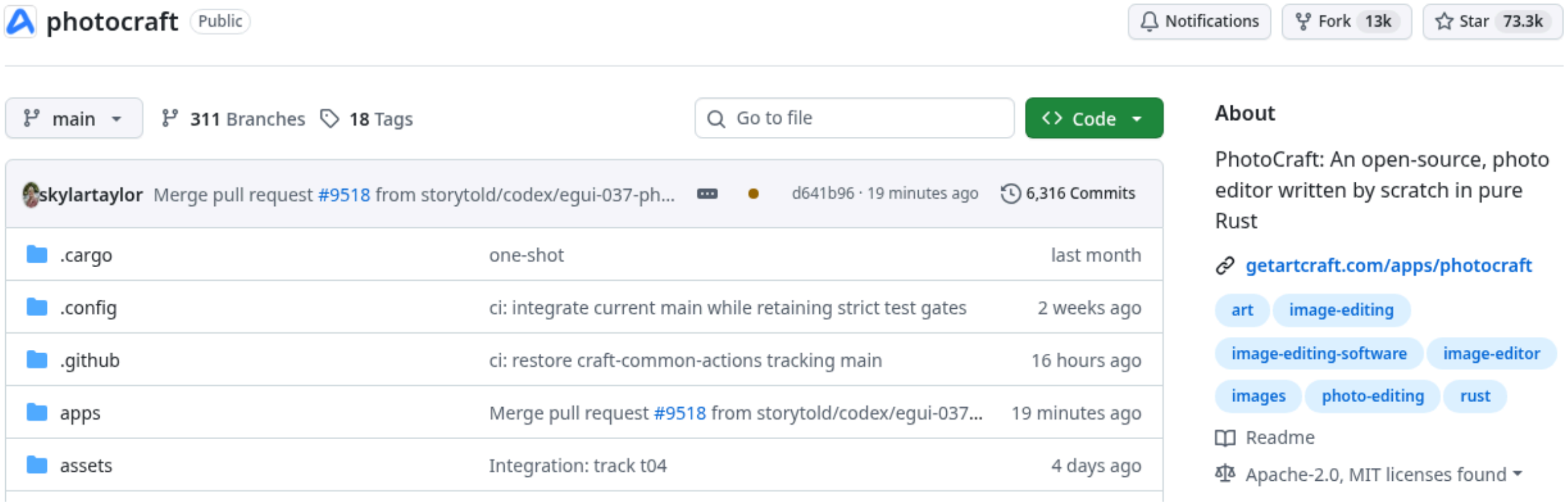
**RESPOSTA**

> *"MIT. This covers the mod's own code only, not Super Mario 64's or Elden Ring's content: Mario's model, textures, sounds and animations come from the player's ROM."*

A licença só dá o que é do autor: o código que ele escreveu. O Mario não é dele. Cada jogador põe na pasta a ROM do próprio cartucho, e o modelo, as texturas e os sons saem de lá.

# E se ninguém olhar o código?

# PhotoCraft, outubro de 2026




Fonte: github.com/storytold/photocraft, 26/10/2026

# Um Photoshop aberto, feito com IA

Um programador, Brandon Thomas, publicou entre 30/09 e 01/10/2026 sete programas que imitam os da Adobe, como Photoshop, Illustrator e Premiere, escritos com o Claude. O de imagem, o PhotoCraft, tinha 73 mil estrelas no GitHub em 26/10.

> *"Clean-room: implemented from public specs and observed behaviour only. No proprietary code, shaders or assets."*

Licença: MIT ou Apache-2.0, à escolha de quem usa.

# Clean room, desde os anos 1980

Para vender computadores compatíveis com o PC da IBM sem copiar o programa de inicialização dela, a BIOS, empresas como a Compaq e a Phoenix dividiram o trabalho em duas equipes.

A primeira estuda o original e escreve uma especificação, sem código. A segunda, que nunca viu o original, programa a partir do texto. Se o resultado parecer com o da IBM, a defesa é que foi criado de forma independente.

A Compaq lançou o seu em 1982; a Phoenix passou a licenciar a BIOS dela em 1984. Quem copiou o código da IBM, como a Eagle e a Corona, fez acordo e reescreveu.

# A licença de vocês

O grupo escolhe uma licença para o repositório. Ela cobre o que o grupo escreveu.

O resto tem dono e licença próprios: as decisões raspadas do tribunal, as bibliotecas instaladas com `uv add` e, a partir de hoje, o modelo de linguagem que o programa vai chamar.

# Cem sentenças para classificar

# Em quantas o pedido foi acolhido?

Uma advogada de consumidor raspou cem sentenças sobre negativa de cobertura de plano de saúde. Ela quer saber em quantas o juiz acolheu o pedido do paciente.

A resposta está no texto de cada sentença, e cada juiz escreve do seu jeito.

# Procurar a palavra "procedente"

```
sentencas = [
    "Julgo procedente o pedido.",
    "Julgo improcedente o pedido.",
    "Julgo parcialmente procedente o pedido.",
]
for texto in sentencas:
    print("procedente" in texto)
```

O `in` que vocês usaram com listas também procura um trecho dentro de um texto. O que aparece?

# O trecho dentro da palavra

**RESPOSTA**

```
True
True
True
```

O `in` acha "procedente" em qualquer lugar do texto: dentro de "improcedente" e de "parcialmente procedente" também.

# Testar "improcedente" primeiro

```
sentencas = [
    "Julgo improcedente o pedido.",
    "Rejeito o pedido da autora.",
    "O pedido não merece acolhida.",
]
for texto in sentencas:
    if "improcedente" in texto:
        print("improcedente")
    elif "procedente" in texto:
        print("procedente")
    else:
        print("?")
```

O que aparece?

# Três sentenças improcedentes, uma reconhecida

**RESPOSTA**

```
improcedente
?
?
```

As três negam o pedido. Duas dizem isso sem a palavra "improcedente", e cada forma nova de escrever pediria mais um `if`. "Parcialmente procedente" cairia em "procedente". A lista de regras não acaba.

# O modelo dentro do programa

# Cem sentenças no chat

Uma sentença, vocês colam no chat e perguntam se o pedido foi acolhido.

Como fariam com as cem? E com dez mil?

# O programa pergunta ao modelo

**RESPOSTA**

O programa manda cada sentença ao modelo sozinho, num `for`. Cada envio é um pedido pela internet a um computador do provedor, que devolve uma resposta:

| NO COMPUTADOR | **pedido** → | NO PROVEDOR | **resposta** → | DE VOLTA |
|---|---|---|---|---|
| O programa | | O modelo | | O texto escrito |

O pedido leva o prompt e a chave; a resposta traz o texto. É isso que o chat faz por trás da tela, e é o que se chama de API.

# O que é uma API

Um mesmo serviço costuma ter duas portas: uma tela para pessoas e um endereço para programas. A porta dos programas é a API: o endereço, a forma do pedido e a forma da resposta são fixos, para que um programa monte o pedido sozinho.

| serviço | para pessoas | para programas |
|---|---|---|
| Claude | o chat, em claude.ai | a API, que o programa chama com a chave |
| Google Maps | o aplicativo do celular | a API, que o Uber chama para traçar a corrida |
| TJSP | a página de consulta | não há; o juscraper lê a página feita para gente |

# Precisamos de uma chave

O Tribunal Fictício respondia a qualquer pedido. O provedor do modelo só responde a pedido que traz uma chave: um código comprido, ligado à conta de quem o criou.

Por que o provedor exige isso, e o tribunal não exigia?

# Por que o provedor pede uma chave

**RESPOSTA**

Cada resposta custa ao provedor tempo de computador. A chave diz quem está pedindo: a conta de quem paga, ou de quem usa a cota gratuita. Sem chave, ou com uma que não vale, o pedido é recusado; com ela, o provedor conta os pedidos da conta e recusa os que passam do limite.

E quem tem a chave usa a conta em nome do dono dela: pedido, cota e cobrança caem nele. Por isso cada um cria a sua.

# Criar a conta e a chave na Groq

| # | O que fazer |
|---|---|
| 1 | Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail |
| 2 | No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1` |
| 3 | Copiar a chave agora: ela começa com `gsk_` |

A chave é sua. Não mandem a ninguém, nem para mim.

# Onde guardar a chave

O jeito mais rápido é colar a chave no próprio programa:

```
chave = "gsk_..."
```

O programa funciona. O que pode dar errado?

# A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário `codigo`.

# A chave vai junto com o arquivo

**RESPOSTA**

O arquivo vai para o repositório, e a chave vai junto. Apagar a linha e enviar de novo não resolve: a versão anterior continua no histórico.

Chave que vazou se apaga no painel da Groq, em **API Keys**, e se troca por outra.

# A chave no arquivo .env

| # | O que fazer, na pasta `cdad1` |
|---|---|
| 1 | No terminal do VS Code: `uv add python-dotenv langchain-groq` |
| 2 | Criar o arquivo `.env`, com uma linha só: `GROQ_API_KEY=` e a sua chave colada depois do = |
| 3 | Abrir o arquivo `.gitignore` e acrescentar uma linha com `.env` |

O `.gitignore` lista o que o Git não registra: o `.env` fica no computador e nunca vai para o GitHub.

# Conferir a chave

```python
import os
from dotenv import load_dotenv

load_dotenv()
chave = os.environ["GROQ_API_KEY"]
print(chave[:4])
```

Rodem num notebook da pasta `cdad1`. O que aparece? E se o `.env` não existisse?

# os.environ lê o ambiente

**RESPOSTA**

```
gsk_
```

O `load_dotenv()` leva o que está no `.env` para o ambiente, e `os.environ` é um dicionário com ele. Sem o arquivo, a chave não existe:

```
KeyError: 'GROQ_API_KEY'
```

Só os quatro primeiros caracteres vão para a tela: a chave inteira nunca aparece num print.

# A primeira chamada

# Trazer as peças e ler o .env

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
```

`import os` traz o módulo de onde vem o `os.environ`. As duas linhas com `from` trazem uma peça só de cada biblioteca instalada com `uv add`: a função `load_dotenv` e a classe `ChatGroq`.

`load_dotenv()` leva a chave do `.env` para o ambiente, como no slide anterior.

# Criar o objeto do modelo

```
modelo = ChatGroq(model="openai/gpt-oss-120b",
                  api_key=os.environ["GROQ_API_KEY"])
```

`ChatGroq` é uma classe, como `Parte`: chamá-la cria um objeto, que fica na variável `modelo`. Ela recebe dois argumentos: `model`, o nome do modelo na Groq, e `api_key`, a chave lida do ambiente.

Até aqui nada saiu do computador: o objeto só guarda com quem falar e com que chave.

# Escrever a pergunta

```
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
```

Um texto comum, numa variável. É o prompt: tudo o que o modelo vai ler. Aqui, a sentença em que o `if` falhou e a pergunta, no mesmo texto.

# Mandar o pedido

```
resposta = modelo.invoke(pergunta)
```

`invoke` é um método do objeto `modelo`. É nesta linha que o pedido sai pela internet, com o prompt e a chave, e o programa para e espera a resposta voltar.

O que volta não é um texto: é um objeto, guardado em `resposta`.

# Ler o texto da resposta

```
print(resposta.content)
```

O objeto `resposta` carrega mais do que o texto: quantos tokens entraram e saíram, qual modelo respondeu. O atributo `content` é o texto que o modelo escreveu.

# Perguntar ao modelo pelo Python

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b",
                  api_key=os.environ["GROQ_API_KEY"])
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
resposta = modelo.invoke(pergunta)
print(resposta.content)
```

Rodem. O que aparece?

# Objeto, método e atributo

**RESPOSTA**

Aparece o texto que o modelo escreveu, e cada um viu o seu.

| No código | O que é |
|---|---|
| `ChatGroq(...)` | cria o objeto do modelo |
| `modelo.invoke(...)` | o método que manda o pedido |
| `resposta` | um objeto, com o texto e os tokens |
| `resposta.content` | o atributo com o texto |

# O pedido e a resposta, passo a passo

Animação no navegador: `animacao-api-llm.html`, cenário `ambiente`.

# Agora, uma sentença da lista

No programa da primeira chamada, troquem a pergunta por estas duas linhas e rodem:

```
pergunta = ("O pedido não merece acolhida. "
            "Procedente ou improcedente?")
```

Para essa sentença, o `if` imprimiu `?`. Cada um manda a palavra que o modelo respondeu, tal como veio.

# O que o modelo leu

**RESPOSTA**

A sentença nega o pedido sem usar nenhuma das duas palavras, e o `if` não tinha regra para ela. As barras mostram o que o modelo respondeu a cada um.

Se as respostas vieram iguais, o modelo fez o que a regra escrita à mão não conseguia. Se vieram diferentes, ou com uma frase em volta da palavra, isso fica como pergunta para quando as cem sentenças passarem pelo programa de uma vez.

# O prompt de sistema

Insper

# O que o Claude recebe antes da sua mensagem?

Vocês abrem o claude.ai e escrevem a primeira mensagem. Ela não é a primeira coisa que o modelo lê.

Apostem: o que chega ao modelo antes dela, e quem escreveu?

# Um texto fixo, que ninguém vê

**RESPOSTA**

Antes de cada conversa, o chat manda ao modelo um texto escrito pela empresa: o prompt de sistema. Ele vai em toda conversa, e o usuário não o vê.

A Anthropic publica o dela. O do Claude Haiku 5.5, de 7 de outubro de 2026, tem cerca de 4.200 palavras: a data de hoje, o que o modelo sabe e até quando, como falar, como formatar a resposta e o que fazer quando não tem certeza.

# O que ele sabe, e até quando

> *"Claude's reliable knowledge cutoff, past which it can't answer reliably, is the end of June 2026. It answers the way a highly informed individual in June 2026 would if talking to someone from {{currentDateTime}}, and can say so when relevant."*

> *"If not certain something it recalls is true and on-point, it says so and suggests enabling web search for newer information."*

O `{{currentDateTime}}` é um buraco que o chat preenche com a data de hoje antes de mandar: por isso o modelo sabe que dia é.

Anthropic, prompt de sistema do Claude Haiku 5.5 em claude.ai, 7 de outubro de 2026

# Como ele fala

*"Claude uses a warm tone, treating people with kindness and without making negative assumptions about their judgment or abilities. Claude is still willing to push back and be honest, but does so constructively, with kindness, empathy, and the person's best interests in mind."*

*"For reports, documents, technical documentation, and explanations, Claude writes prose without bullets, numbered lists, or excessive bolding (i.e. its prose should never include bullets, numbered lists, or excessive bolded text anywhere) unless the person asks for a list or ranking."*

Anthropic, prompt de sistema do Claude Haiku 5.5 em claude.ai, 7 de outubro de 2026

# Vale para o chat, não para a API

A Anthropic avisa na mesma página: esses prompts valem para o site e os aplicativos. Pela API não vai nenhum.

Quem chama o modelo pelo programa escreve o próprio prompt de sistema, ou não manda nenhum. Na primeira chamada de hoje, não mandamos: foi só a pergunta.

No pedido, cada texto vai com um papel: o prompt de sistema é a mensagem `system` o que o usuário escreve é a mensagem `user`.

# A instrução separada da sentença

```
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b")
instrucao = "Uma palavra só: PROCEDENTE ou IMPROCEDENTE."
texto = "O pedido não merece acolhida."
mensagens = [("system", instrucao), ("user", texto)]
resposta = modelo.invoke(mensagens)
print(resposta.content)
```

O que mudou em relação à primeira chamada?

# Dois papéis, um pedido

**RESPOSTA**

O `invoke` recebe uma lista de mensagens, cada uma um par (papel, texto), em vez de um texto só. O pedido que saiu levou as duas, nesta ordem:

```
system: "Uma palavra só: PROCEDENTE ou IMPROCEDENTE."
user: "O pedido não merece acolhida."
```

A instrução vai em `system`, e a sentença, em `user`. E o `api_key` saiu: sem ele, `ChatGroq` lê `GROQ_API_KEY` do ambiente sozinho.

# As três sentenças de uma vez

As quatro primeiras linhas são as do programa anterior. Daí em diante:

```
instrucao = "Uma palavra só: PROCEDENTE ou IMPROCEDENTE."
sentencas = ["Julgo improcedente o pedido.",
             "Rejeito o pedido da autora.",
             "O pedido não merece acolhida."]
for texto in sentencas:
    mensagens = [("system", instrucao), ("user", texto)]
    resposta = modelo.invoke(mensagens)
    print(resposta.content)
```

Quantos pedidos saem? O que muda de um para o outro?

# Três pedidos, um por volta

**RESPOSTA**

O `for` faz 3 pedidos, um por sentença. A instrução vai igual nos três; só a mensagem `user` muda.

É o programa que a advogada precisava: com cem sentenças, a lista cresce e o `for` continua o mesmo. O que fazer com cem respostas, e como garantir que cada uma vem só com a palavra, é a aula em que elas entram numa tabela.

# Temis: como venceu a fase 1

Insper

# Temis, vencedora da fase 1




Um assistente que lê o aviso de andamento do processo no e-mail, calcula o prazo e põe na agenda.

O grupo conta como se dividiu, que referências buscou e que ferramentas usou. Depois, as perguntas são de vocês.

# Fase 2: banca, demonstração e nota

Insper

# A banca da fase 2

## Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

# A banca da fase 2

## Quem avalia

Pessoas que trabalham com tecnologia. Os nomes vão por e-mail quando estiverem confirmados.

## O que costuma pesar

Em bancas de tecnologia: se o produto funciona de verdade; se aguenta erro; se o código é legível; se o grupo sabe explicar o que o assistente faz.

# A demonstração

| Quando | O quê |
|---|---|
| Até 17/11 | um vídeo do produto funcionando, enviado com o endereço do repositório |
| Em 18/11 | o pitch, como na fase 1: slides e demonstração do jeito que o grupo preferir |

# A tier list com dez grupos

| Faixa | S | A | B | C | D | F |
|---|---|---|---|---|---|---|
| Vagas | 1 | 2 | 2 | 2 | 2 | 1 |
| Pontos | 15 | 12 | 9 | 6 | 3 | 1 |

As mesmas faixas e os mesmos pontos da fase 1. São dez vagas para dez grupos: nenhuma faixa leva um grupo a mais.

# A nota da fase 2

| Posição | Nota |
|---|---|
| 1ª e 2ª | 1,5 |
| 3ª a 6ª | 1,4; 1,3; 1,2; 1,1 |
| 7ª a 10ª | de 0,1 a 1,0, pelo repositório |

# Na próxima aula

**Como o modelo deixa de só responder e passa a agir: um programa que dá ferramentas ao modelo, roda em loop até cumprir o objetivo e pede permissão antes de mexer no que importa.**

Até o início dela, quarta, 28/10: a sua chave da Groq funcionando, com a primeira chamada respondendo no seu computador.